Use Case: Forget Password

Iteration: 1

Primary Actor: User

Goal in Context: Allow users that have forgotten their passwords to reset their account password. Users must be able to prove they own and are authenticated into the account, by entering details that were used when signing up. This can include but not limited to: Name, location of account creation, security questions, and the pertaining username of the account.

Preconditions:

1. User must have an account in the database to recover
2. User must be authenticated to recover the password, be the original owner and have full access to their data
3. Users must have relevant details which are needed to authenticate if the owner of the account is the individual attempting to recover the password.
4. Reset password meets the criteria, and is valid to be renewed for the account that is being recovered.
5. All details of the account must not change during password recovery, which may create conflicts when verifying data.

Trigger:

1. **Signing Up**: User has entered a name and username that already exists on the database, user is redirected to forget-password to recover their account.
2. **Login:** Users cannot authenticate into their account and must recover their account by correctly entering partial details.

Scenario:

1. User executes the program using the integrated development environment or through an executable.
2. User selects login option from the landing page
    a. User cannot login to their account and enters the forget-password option
    b. User enters the name, username, and necessary details to recovery account (note based on account familiarity)
3. User selects signup and enters an username that exists on the database

    a. User enters the name, username, and necessary details to recovery account (note based on account familiarity)
4. User receives confirmation, and is able to login to the system using their renewed credentials
5. User selects login option and enters necessary details (see use case)
6. User proceeds to the player profile

Post Conditions:

1. The valid password that is entered into the text field is the new allocated password for the given account.
2. Users are able to use the new password to login to the system, and all instances have been replaced.
3. Users have recovered their previous save states, including profile customization and leaderboards stats.
4. Users are able to experience all program functions that ordinary new/existing users have access to.

Exceptions:

1. User has not been authenticated to change password, the program returns the user back to the landing page, and password recovery attempts on the account will be noted down. Users who have logged into the account will have a system prompt to show the unrecognized system recovery.
2. New password does not meet the requirements set by the system, the program will alert the user that the password does not meet criteria and will need to retry.
3. Password recovery attempt on the account has exceeded the set limit, the user will be logged out and will need to continue by creating a new account.
4. User abandons password recovery while filling out required text field, program will redirect user back to the landing page, where user can either choose to login or sign up (see use cases)

Priority: High, cannot access other features without logging in.

When available: First-second iteration

Frequency of Use: Once during the program lifecycle, unless requiring a restart.

Channel to Actor: Program landing page.

Secondary Actor: Backend/network Request

Channel to Secondary Actor: In special cases, where a user cannot be authenticated into the system, a backend/network request can be sent to inspect details and manually verify the user.

Open Issues:

1. Currently login does not accommodate for utilities that are used by forgot-password (i.e. security questions), which may need to be revised after the initial iteration to allow for complex user interactions.
2. User verification must be tested thoroughly to allow access only to account owners, and not any unauthorized individuals which are attempting to login fraudulently.
3. Need to set a limit of password changes to ensure user security is not compromised when changing password, at some limit users may need to create a new account.
4. Security questions which will be added after the first iteration need to be representative of the user, in a way that uniqueness allows for secure encryption.
5. Users may forget their passwords overtime, need to devise a way to reset passwords after a certain duration or routinely after the user has finished a game session.